



Stage de pré-rentree — Entrée en Terminale, NSI

Adam Zahouani · Terminale · NSI · 2026-08-13

Durée non mesurée — saisie papier.

Ce bilan fait le point, domaine par domaine : ce qui est solide, ce qui se consolide, ce qui se corrige — et l'ordre dans lequel le stage va s'y prendre. Sur les sept domaines évalués : un acquis, trois à installer, trois à rectifier en priorité.

Comment lire ce bilan

Ce bilan ne donne pas de note. Il croise deux informations : ce que tu réussis, et la confiance avec laquelle tu réponds. C'est ce croisement qui distingue un point solide, un point à consolider, une notion à installer — et une certitude à revoir. Les certitudes à revoir passent en premier : tant qu'on croit juste une idée fausse, on ne la corrige pas seul.

Ta carte maîtrise × confiance

Réponses sûres... mais à revoir

Types construits · Programmation · Données en tables

Priorité : on confronte, puis on reconstruit.

Réponses justes, données avec confiance

Architecture et systèmes

Acquis : on entretient.

Difficulté repérée, sans fausse certitude

Représentation binaire · Booléens et logique
· Algorithmique

Prêt à apprendre : on installe.

Réponses justes, mais hésitantes

Rien ici : quand tu réussis, tu le sais. Ta confiance est bien calée sur tes réussites.

Tes repères en chiffres

18_{sur 18}

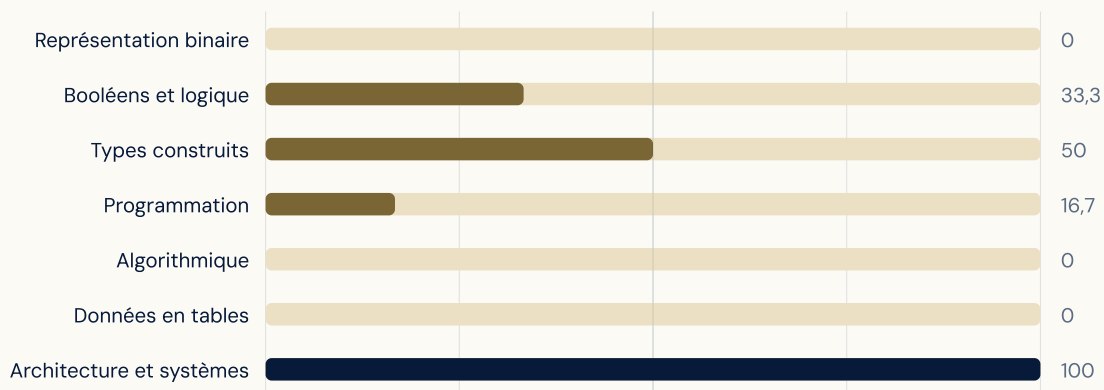
questions traitées

67%

des cas où tu sais dire si ta réponse est sûre —
c'est ta boussole de révision

1 domaine solide (14 %) 3 domaines à installer (43 %) 3 domaines à rectifier en priorité (43 %)

Ta réussite par domaine



Ces chiffres décrivent ta réussite domaine par domaine — ce bilan ne donne pas de note.

Tes points d'appui

1. Architecture et systèmes — réponses justes, données avec assurance : c'est un vrai point d'appui pour la suite.

Tes priorités pour le stage

1. Données en tables — une certitude à revoir avant d'avancer. C'est le travail le plus rentable des premières séances.
2. En programmation, tu as répondu avec certitude... et la réponse était fausse. Rien de grave : on te montrera précisément où le raisonnement bifurque.
3. Types construits — une réponse fausse a été donnée avec assurance. C'est la priorité : une conviction erronée ne se corrige qu'une fois qu'on l'a vue.
4. Représentation binaire — quelque chose manque, et tu le sais déjà : c'est une vraie lucidité. Il reste à installer les bons repères.
5. Algorithmique — une notion à construire plutôt qu'à corriger. Exactement le type de lacune qu'un stage comble vite.
6. Ce qui manque sur les booléens est identifié. Aucune fausse certitude à défaire : on installe, puis on entraîne.

Ton parcours pendant le stage — cinq séances

SÉANCE 1 · DEUX HEURES

Données en tables [CONFRONTER](#)

Objectif : Rectifier une certitude erronée sur les données en tables.

Démarche : Partir d'un cas qui met la conviction en défaut, faire verbaliser le raisonnement, puis reconstruire la notion avant tout entraînement.

Programmation [CONFRONTER](#)

Objectif : Faire apparaître, puis lever, l'idée fausse installée en programmation.

Démarche : Provoquer le conflit cognitif sur un exemple choisi, comparer les deux raisonnements, puis stabiliser la version juste.

SÉANCE 2 · DEUX HEURES

Types construits [CONFRONTER](#)

Objectif : Reconstruire une notion tenue pour acquise sur les types construits.

Démarche : Faire verbaliser la règle utilisée, la confronter à un contre-exemple, puis reconstruire pas à pas.

SÉANCE 3 · DEUX HEURES

Représentation binaire [INSTALLER](#)

Objectif : Installer les repères indispensables en représentation binaire.

Démarche : Poser les définitions utiles, montrer des exemples résolus, puis proposer un entraînement court et répété.

SÉANCE 4 · DEUX HEURES

Algorithmique [INSTALLER](#)

Objectif : Poser les définitions et les gestes de base en algorithmique.

Démarche : Avancer du simple vers le composé : un repère, un exemple, un essai — puis répéter.

SÉANCE 5 · DEUX HEURES

Booléens et logique

INSTALLER

Objectif : Construire les premiers automatismes sur les booléens.

Démarche : Installer la notion par petites étapes vérifiées, avant un entraînement bref à la séance suivante.

Ton plan d'action entre les séances

1. D'ici la première séance, repère une question sur les données en tables dont la réponse te paraît évidente : on la mettra à l'épreuve ensemble.
2. Note deux ou trois réflexes que tu utilises en programmation : les écrire permettra de voir lequel bifurque.
3. Ne cherche pas à « réviser plus » sur les types construits : viens avec tes certitudes, ce sont elles qu'on va tester.
4. Et pendant tout le stage, garde le réflexe du bilan : avant de valider une réponse, demande-toi « j'en suis sûr, ou je crois l'être ? ».

Ton auto-évaluation

Point fort : ton auto-évaluation est fiable — tu sais globalement ce que tu sais. C'est un vrai atout pour réviser juste, sans perdre de temps.

Le détail de tes réponses

Question par question : ta réponse, la réponse attendue quand elles diffèrent, et ce qu'il faut en retenir. C'est ta meilleure base de révision d'ici la première séance.

Représentation binaire

À REVOIR Certitude déclarée : 2/4 — « peu sûr »

Quelle est l'écriture binaire de l'entier 13 ?

Réponse donnée : 1110

Réponse attendue : 1101

D'où vient l'erreur : Vaut 14 : décale d'une unité.

Ce qu'il faut retenir : $13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0$, ce qui s'écrit 1101 en binaire.

À REVOIR Certitude déclarée : 1/4 — « je devine »

Que vaut l'entier hexadécimal 0x1F en base 10 ?

Réponse donnée : 25

Réponse attendue : 31

D'où vient l'erreur : Traite F comme s'il valait 9.

Ce qu'il faut retenir : $0x1F = 1 \times 16 + 15 = 31$, car en hexadécimal F représente 15.

Booléens et logique

JUSTE Certitude déclarée : 3/4 — « plutôt sûr »

En Python, que vaut l'expression (True and not False) ?

Réponse donnée : True

Ce qu'il faut retenir : not False vaut True, et True and True vaut True.

À REVOIR Certitude déclarée : 2/4 — « peu sûr »

L'expression « A OU B » est fausse uniquement lorsque :

Réponse donnée : A est faux et B est vrai

Réponse attendue : A et B sont tous les deux faux

D'où vient l'erreur : A OU B est alors vrai grâce à B.

Ce qu'il faut retenir : Une disjonction n'est fausse que si aucun des deux opérandes n'est vrai.

Types construits

JUSTE Certitude déclarée : 4/4 — « certain »

En Python, on définit L = [10, 20, 30, 40]. Que vaut L[1] ?

Réponse donnée : 20

Ce qu'il faut retenir : Les listes sont indexées à partir de 0 : L[0]=10, L[1]=20.

JUSTE Certitude déclarée : 4/4 — « certain »

Après `L = [1, 2, 3]` puis `L.append(4)`, que contient L ?

Réponse donnée : [1, 2, 3, 4]

Ce qu'il faut retenir : `append(4)` ajoute 4 à la fin : L devient [1, 2, 3, 4].

À REVOIR Certitude déclarée : 4/4 — « certain »

On définit `d = {'a': 1, 'b': 2}`. Que vaut `d['b']` ?

Réponse donnée : une erreur

Réponse attendue : 2

D'où vient l'erreur : Suppose à tort que la clé 'b' n'existe pas.

Ce qu'il faut retenir : `d['b']` renvoie la valeur associée à la clé 'b', soit 2.

À REVOIR Certitude déclarée : 4/4 — « certain »

Comment ajouter au dictionnaire `d` la clé 'c' associée à la valeur 3 ?

Réponse donnée : `d.append('c', 3)`

Réponse attendue : `d['c'] = 3`

D'où vient l'erreur : `append` est une méthode des listes, pas des dictionnaires.

Ce qu'il faut retenir : On crée ou modifie une entrée par affectation : `d['c'] = 3`.

Programmation

JUSTE Certitude déclarée : 4/4 — « certain »

Soit la fonction : `def f(x): return x * 2`. Que renvoie `f(5)` ?

Réponse donnée : 10

Ce qu'il faut retenir : `f(5)` calcule `5 * 2` et renvoie 10.

À REVOIR Certitude déclarée : 2/4 — « peu sûr »

En Python, que renvoie une fonction qui ne comporte aucune instruction `return` ?

Réponse donnée : une erreur

Réponse attendue : `None`

D'où vient l'erreur : Une fonction sans `return` est valide ; elle ne provoque pas d'erreur.

Ce qu'il faut retenir : Sans `return`, une fonction renvoie implicitement la valeur `None`.

À REVOIR Certitude déclarée : 3/4 — « plutôt sûr »

Combien d'itérations effectue la boucle « `for i in range(3):` » ?

Réponse donnée : de 1 à 3

Réponse attendue : 3

D'où vient l'erreur : `range(3)` commence à 0, pas à 1.

Ce qu'il faut retenir : `range(3)` produit les valeurs 0, 1, 2 : la boucle s'exécute 3 fois.

À REVOIR Certitude déclarée : 3/4 — « plutôt sûr »

Que vaut s après : s = 0 puis « for i in range(1, 4): s = s + i » ?

Réponse donnée : 10

Réponse attendue : 6

D'où vient l'erreur : Inclut à tort la valeur 4 dans la somme.

Ce qu'il faut retenir : range(1, 4) donne 1, 2, 3 ; la somme cumulée vaut 1 + 2 + 3 = 6.

Algorithmique

À REVOIR Certitude déclarée : 3/4 — « plutôt sûr »

La recherche dichotomique ne fonctionne que si le tableau est :

Réponse donnée : de taille paire

Réponse attendue : trié

D'où vient l'erreur : La dichotomie fonctionne quelle que soit la parité de la taille.

Ce qu'il faut retenir : La recherche dichotomique compare à l'élément central : elle exige un tableau trié.

À REVOIR Certitude déclarée : 1/4 — « je devine »

Dans un tableau trié de 16 éléments, le nombre de comparaisons d'une recherche dichotomique est de l'ordre de :

Réponse donnée : 16

Réponse attendue : 4 ($\log_2$ de 16)

D'où vient l'erreur : Correspond à une recherche séquentielle, pas dichotomique.

Ce qu'il faut retenir : La dichotomie divise la taille par deux à chaque étape : $\log_2(16) = 4$ comparaisons environ.

Données en tables

À REVOIR Certitude déclarée : 2/4 — « peu sûr »

Dans une table de données (type CSV), une ligne représente en général :

Réponse donnée : un attribut

Réponse attendue : un enregistrement (un individu ou objet)

D'où vient l'erreur : L'attribut correspond à une colonne, pas à une ligne.

Ce qu'il faut retenir : Dans une table, chaque ligne est un enregistrement, chaque colonne un attribut.

À REVOIR Certitude déclarée : 3/4 — « plutôt sûr »

Ne garder que les lignes d'une table vérifiant « âge > 18 » est une opération de :

Réponse donnée : tri

Réponse attendue : sélection (filtrage des lignes)

D'où vient l'erreur : Le tri réordonne les lignes sans en supprimer.

Ce qu'il faut retenir : Choisir les lignes selon une condition est une sélection (ou filtrage) ; choisir des colonnes serait une projection.

Architecture et systèmes

JUSTE Certitude déclarée : 3/4 — « plutôt sûr »

Dans le modèle de von Neumann, la partie qui effectue les calculs est :

Réponse donnée : l'unité arithmétique et logique (UAL)

Ce qu'il faut retenir : Dans le processeur, l'UAL réalise les opérations arithmétiques et logiques.

JUSTE Certitude déclarée : 3/4 — « plutôt sûr »

Quel est le rôle principal d'un système d'exploitation ?

Réponse donnée : gérer les ressources matérielles et l'exécution des programmes

Ce qu'il faut retenir : Le système d'exploitation gère mémoire, processeur, périphériques et l'exécution des programmes.

Pour finir

Ce bilan n'est pas une note : c'est une carte. Elle dit où appuyer l'effort pour que l'année démarre du bon pied.

